

计算机网络课程设计 任务书

基于 Socket 的自定义协议聊天室
实现与网络分析

2026 年 6 月

1 课程基本信息

- 课程名称：计算机网络课程设计
- 参考教材：James F. Kurose, Keith W. Ross, 《计算机网络：自顶向下方法》（第 8 版）
- 先修课程：计算机网络、程序设计基础 (Python/Java/C)

对应教材章节：

- 第 1 章——时延、吞吐量等性能指标
- 第 2 章——Socket 编程、应用层协议设计（核心）
- 第 3 章——TCP 可靠传输、三次握手、四次挥手（抓包分析）

2 设计目标

本课程设计旨在通过实现一个基于 TCP Socket 的自定义协议聊天室，帮助学生达成以下目标：

1. 深刻理解应用层协议的设计原理，掌握自定义协议的报文格式和交互过程；
2. 熟练掌握 TCP Socket 编程，理解 socket API 与底层 TCP 协议栈的关系；
3. 掌握使用 Wireshark 抓包分析网络协议的方法；
4. 理解并发模型在网络服务中的应用；
5. 培养设计协议 → 实现协议 → 验证协议的工程实践能力。

3 预备知识

在开始本课程设计之前，学生应熟练掌握以下知识点：

1. TCP Socket 编程基础：socket()、bind()、listen()、accept()、connect()、recv()、send()、close() 的用法
2. TCP 协议基础：三次握手、四次挥手、序列号与确认号
3. 应用层协议设计：报文结构、头部字段、消息边界
4. Python 基础：字符串与字节流操作、文件 I/O、异常处理、多线程编程
5. Wireshark 基础：设置过滤规则、追踪 TCP 流、查看数据包各字段

4 任务内容与评分标准

本课设采用分层递进的任务结构。前三层为必做任务，共 70 分；扩展任务为选做，共 30 分。

每个功能点均须同时提交可运行代码、测试截图或抓包截图、报告中的实现说明。答辩时以现场可复现的运行结果和报告中可定位的代码说明为主要依据；仅有界面截图但不能运行，或仅有代码但不能说明测试过程，均不能视为完整完成。

4.1 L1. 第一层：CHAT/1.0 核心服务器（30 分）

任务要求：

基于 Python 标准库中的 `socket` 模块，实现一个支持自定义协议 CHAT/1.0 的聊天服务器。服务器能够接受客户端连接，解析 CHAT/1.0 协议报文，管理用户登录状态，实现消息广播转发。

关于消息边界的重要说明：

TCP 是流式协议，不维护应用层的消息边界。一次 `recv()` 可能收到多个完整的请求（粘包），也可能只收到半个请求（半包）。你的实现必须正确处理消息边界：建议维护一个接收缓冲区，循环从中提取完整的请求。请求的边界由 `\r\n\r\n`（头部结束标记）和 `Content-Length`（消息体长度）共同确定。

需要实现的具体功能点：

1. TCP 连接管理（5 分）

- 创建 `welcome socket`，绑定指定端口（如 8888），开始监听，并在启动日志中输出监听地址和端口
- 正确调用 `accept()` 接受客户端连接，获取 `connection socket`；至少能接受两个客户端依次连接
- 客户端发送 `LOGOUT` 或断开连接后，服务器能关闭对应 `connection socket`，释放资源，且不影响其他已连接客户端

2. CHAT/1.0 请求解析（8 分）

- 从 `connection socket` 读取客户端发送的请求数据（注意消息边界处理）
- 解析命令行（命令、协议版本）
- 解析请求头部字段（至少能解析 `Content-Length`；对于 `LOGIN` 命令还需解析 `User` 头部获取用户名）
- 验证命令类型：L1 阶段只接受 `LOGIN`、`MSG`、`LIST`、`LOGOUT` 四种命令，未知命令返回 `CHAT/1.0 400 BAD_REQUEST`
- 验证协议版本（只接受 `CHAT/1.0`），版本错误返回 `CHAT/1.0 400 BAD_REQUEST`

- Content-Length 是所有请求的必需头部，缺失或值不是非负整数时返回 400 BAD_REQUEST；若 body 长度不足则保持等待直到数据到齐；不得把半包误当作完整消息处理
- CHAT/1.0 协议格式参见附录 A.1

3. 响应构造与消息转发 (10 分)

- 根据不同的命令构造正确的响应报文：
 - LOGIN 成功：返回 CHAT/1.0 200 OK，响应中包含 Content-Length: 0
 - MSG 接收确认：向发送方返回 CHAT/1.0 200 OK，响应中包含 Content-Length: 0
 - MSG 转发：将消息广播给所有其他已登录用户，转发报文包含 From 和 Content-Length
 - LIST：返回在线用户列表 CHAT/1.0 201 USERLIST，body 为逗号分隔的用户名列表，包含所有已登录用户(含请求者本人)(如 alice,bob,charlie)
 - LOGOUT：返回 CHAT/1.0 200 OK 后关闭连接，并从在线列表移除该用户
- 响应报文格式参见附录 A.2，需实现的状态码参见附录 A.3

4. 用户状态管理 (7 分)

- 维护每个连接的状态（未登录、已登录）
- 未登录状态下，只接受 LOGIN 命令，其他命令返回 401
- 登录时检测用户名是否已被占用，若被占用返回 409
- 客户端断开连接后，自动从在线列表移除；后续 LIST 查询不得再包含该用户
- 用户登录、LOGOUT 或异常断开时，向所有仍在线用户推送更新后的 CHAT/1.0 201 USERLIST 报文，body 为最新用户列表
- 用户名长度 1-20 字符，仅允许字母、数字和下划线([a-zA-Z0-9_])，不区分大小写。User 头部缺失、用户名为空或包含非法字符时返回 400 BAD_REQUEST

关于测试输出中可能出现额外推送的说明：

新用户登录或用户退出时，服务器会向其他在线客户端主动推送 CHAT/1.0 201 USERLIST 报文。这意味着你的 recv() 在一次调用中可能收到多种报文（推送的用户列表 201 USERLIST、命令响应 200 OK、转发消息 202 MSG_RELAYED 等），它们可能交错到达。

你需要自行设计方式将异步推送与命令响应区分开——例如在接收循环中解析每个报文的 状态码，根据状态码决定如何处理。这是一个在实际网络编程中必须面对的经典问题。

注意：以下三个测试脚本只演示如何构造和发送测试请求，不负责替你区分异步推送和命令响应。你的客户端或测试程序需要自行解析收到的 CHAT/1.0 报文。

★ **测试点 1：**架好一个基础聊天服务器，并使用 Python socket 脚本至少完成以下三类测试。

测试 1：基本功能测试。启动服务器后，下面的 Python 脚本可用于构造两个客户端 alice 和 bob，完成登录、广播消息、查询在线列表和退出。

```

1 import socket
2 import time
3
4 def recv_print(name, sock):
5     data = sock.recv(4096)
6     print(name, data.decode(errors="replace"))
7
8 alice = socket.socket()
9 alice.connect(("localhost", 8888))
10 bob = socket.socket()
11 bob.connect(("localhost", 8888))
12
13 alice.sendall(b"LOGIN CHAT/1.0\r\nUser: alice\r\nContent-Length: 0\r\n\r\n")
14 recv_print("alice login:", alice)
15 bob.sendall(b"LOGIN CHAT/1.0\r\nUser: bob\r\nContent-Length: 0\r\n\r\n")
16 recv_print("bob login:", bob)
17
18 body = b"Hello World!"
19 req = b"MSG CHAT/1.0\r\nContent-Length: " + str(len(body)).encode()
20 req += b"\r\n\r\n" + body
21 alice.sendall(req)
22 recv_print("alice msg response:", alice)
23 recv_print("bob relayed:", bob)
24
25 bob.sendall(b"LIST CHAT/1.0\r\nContent-Length: 0\r\n\r\n")
26 recv_print("bob list before logout:", bob)
27
28 alice.sendall(b"LOGOUT CHAT/1.0\r\nContent-Length: 0\r\n\r\n")
29 recv_print("alice logout:", alice)
30 alice.close()
31 time.sleep(0.2)

```

```

32
33 bob.sendall(b"LIST CHAT/1.0\r\nContent-Length: 0\r\n\r\n")
34 recv_print("bob list after alice logout:", bob)
35 bob.sendall(b"LOGOUT CHAT/1.0\r\nContent-Length: 0\r\n\r\n")
36 bob.close()

```

预期结果:

- alice 和 bob 登录均返回 CHAT/1.0 200 OK
- alice 发送 MSG 后收到 200 OK
- bob 收到 CHAT/1.0 202 MSG_RELAYED, 其中 From: alice, body 为 Hello World!
- LIST 返回的用户列表包含当前在线用户; LOGOUT 后该用户不再出现在在线列表中
- 用户登录或退出时, 其他在线客户端会收到 CHAT/1.0 201 USERLIST 推送

测试 2: 一条请求分多次发出。下面的 Python 脚本可用于验证服务器不会把一次 `recv()` 误认为一条完整请求。

```

1 import socket
2 import time
3
4 alice = socket.socket()
5 alice.connect(("localhost", 8888))
6 bob = socket.socket()
7 bob.connect(("localhost", 8888))
8
9 alice.sendall(b"LOGIN CHAT/1.0\r\nUser: alice\r\nContent-Length: 0\r\n\r\n")
10 print("alice login:", alice.recv(4096).decode(errors="replace"))
11 bob.sendall(b"LOGIN CHAT/1.0\r\nUser: bob\r\nContent-Length: 0\r\n\r\n")
12 print("bob login:", bob.recv(4096).decode(errors="replace"))
13
14 part1 = (
15     "MSG CHAT/1.0\r\n"
16     "Content-Length: 11\r\n"
17     "\r\n"
18     "hello"
19 ).encode()

```

```

20 part2 = " world".encode()
21
22 alice.sendall(part1)
23 time.sleep(0.2)      # 故意停顿，使服务器更容易先收到不完整请求
24 alice.sendall(part2)
25
26 print("alice response:", alice.recv(4096).decode(errors="replace"))
27 print("bob relayed:", bob.recv(4096).decode(errors="replace"))

```

预期结果：服务器应该能够完整读取间断消息，其他在线客户端最终应收到完整转发消息 hello world，不能只收到 hello，也不能返回错误。

测试 3：多条请求一次发出。该测试用于验证服务器能从同一段 TCP 字节流中连续解析多条 CHAT/1.0 请求。下面的 Python 脚本把三条完整的 MSG 请求拼接成一个字节串，并且只调用一次 sendall()。

```

1  import socket
2
3  alice = socket.socket()
4  alice.connect(("localhost", 8888))
5  bob = socket.socket()
6  bob.connect(("localhost", 8888))
7
8  alice.sendall(b"LOGIN CHAT/1.0\r\nUser: alice\r\nContent-Length: 0\r\n\r\n")
9  print("alice login:", alice.recv(4096).decode(errors="replace"))
10 bob.sendall(b"LOGIN CHAT/1.0\r\nUser: bob\r\nContent-Length: 0\r\n\r\n")
11 print("bob login:", bob.recv(4096).decode(errors="replace"))
12
13 req1 = "MSG CHAT/1.0\r\nContent-Length: 3\r\n\r\none"
14 req2 = "MSG CHAT/1.0\r\nContent-Length: 3\r\n\r\ntwo"
15 req3 = "MSG CHAT/1.0\r\nContent-Length: 5\r\n\r\nonethree"
16
17 alice.sendall((req1 + req2 + req3).encode())
18
19 print("alice responses:", alice.recv(4096).decode(errors="replace"))
20 print("bob relayed:", bob.recv(4096).decode(errors="replace"))

```

预期结果：服务器应识别并处理三条独立请求。发送方应收到三次 200 OK；其他在线客户端应依次收到 one、two、three 三条转发消息。服务器不能只处理第一条，也不能把 onetwothree 当成一条消息。

上述三类测试结果截图保存，用于撰写报告。

4.2 L2. 第二层：私聊与群聊（25 分）

任务要求：

在 L1 广播聊天功能正确的基础上，扩展 CHAT/1.0 协议，使服务器支持一对一私聊和基于群组的聊天。

具体功能点：

1. 私聊消息（10 分）

- 扩展 MSG 命令，支持 To 头部指定接收者
- 私聊消息只转发给 To 指定的在线用户，不发送给其他在线用户
- 目标用户不存在或未在线时，返回 CHAT/1.0 404 USER_NOT_FOUND
- To 头部缺失时，保持 L1 的全体广播行为
- To 值为发送者自己时，服务器仍按私聊逻辑转发给自己（等同于向自己发送一条消息）

2. 群组管理（8 分）

- 新增 CREATEGROUP 命令，通过 Group 头部指定要创建的群组；创建者自动成为该群组成员，无需额外发送 JOINGROUP
- 新增 JOINGROUP 命令，通过 Group 头部指定要加入的群组
- 服务器维护群组到成员用户的映射关系
- 重复创建群组返回 409 GROUP_EXISTS；加入不存在群组返回 404 GROUP_NOT_FOUND；未登录创建或加入群组返回 401 NOT_LOGGED_IN

3. 群聊消息路由（7 分）

- 使用 To: #group_name 将消息发送到指定群组
- 群聊消息只转发给该群组成员，不转发给未加入该群组的在线用户
- 发送者未加入目标群组时，服务器拒绝发送并返回 403 NOT_GROUP_MEMBER

★ 测试点 2：验证私聊和群聊的消息路由是否正确。

- alice、bob、charlie 登录后，alice 向 bob 发送私聊消息；bob 收到，charlie 不应收到
- alice 向不存在的用户发送私聊消息；服务器返回 404 USER_NOT_FOUND
- alice 创建群组 room1，alice 和 bob 加入，charlie 不加入
- alice 向 room1 发送群聊消息；bob 收到，charlie 不应收到

4.3 L3. 第三层: Wireshark 抓包分析 (15 分)

任务要求:

使用 Wireshark 抓取你自己的聊天服务器与客户端之间的通信数据包,对 CHAT/1.0 协议和 TCP 协议交互过程进行深入分析,撰写分析报告。Wireshark 的基本抓包步骤参见附录 A.7。

需抓包分析的内容:

1. TCP 三次握手分析 (5 分)

- 截取三次握手的三个数据包 (SYN、SYN-ACK、ACK)
- 标注每个包的序列号 (seq) 和确认号 (ack) 的变化
- 说明三次握手解决了什么问题

2. CHAT/1.0 请求-响应分析 (5 分)

- 截取客户端发出的 LOGIN 和 MSG 请求报文,标注命令、头部和消息体
- 截取服务器返回的 200 OK 响应报文和 202 MSG_RELAYED 推送报文
- 将抓包内容与你的代码实现进行对照——你的代码在哪个环节构造/解析了这些内容?

3. TCP 序列号与确认号追踪 (3 分)

- 选取一段连续的数据传输过程,追踪 seq 和 ack 的递增关系
- 计算本次传输的数据总量,验证与 seq 变化量是否一致

4. TCP 连接终止分析 (2 分)

- 截取四次挥手的数据包 (FIN、ACK)
- 标注挥手过程,说明为什么是四次而非两次

4.4 扩展任务 (选做, 共 30 分)

扩展任务可自由选择。

4.4.1 E1: 图形化聊天客户端 (10 分)

- 实现一个图形化聊天客户端,界面需调用自己实现的 CHAT/1.0 客户端协议代码,不得只做静态界面。
- E1 至少需要完整支持 L1 的登录、广播聊天、接收服务器转发消息、在线用户列表显示与更新、退出或关闭连接。L2 的私聊与群聊功能不要求在图形界面中实现。

- 基本界面要求：
 - 登录界面：输入用户名、服务器地址和端口
 - 聊天主界面：消息显示区域、消息输入框、发送按钮
 - 在线用户列表显示区
 - 新消息到达时自动滚动到最底部
- 客户端能够持续接收服务器转发的聊天消息和 USERLIST 推送，界面不会因为等待网络数据而卡死。
- 至少使用两个客户端窗口完成登录、广播发消息、收消息、在线用户列表更新和退出测试。

4.4.2 E2：消息加密（10 分）

- 实现消息体的加密传输，防止明文消息在传输过程中被 Wireshark 直接窃听。
- 加密方式自选，以下提供参考思路：
 - 标准方案——服务器中转加密：客户端在 LOGIN 时通过 Key 头部提交一个密钥（如 Key: mysecret），服务器保存每个用户对应的密钥。发送消息时，发送方客户端用自己的密钥加密消息体发送给服务器，服务器解密后使用接收方的密钥重新加密，再转发给接收方。这样传输过程中任何一方看到的都是密文。
 - 自定义简单算法：如 XOR 加密（每个字节与密钥异或）、凯撒密码（字符偏移）、维吉尼亚密码等。
 - 哈希派生：使用 hashlib 从密钥派生加密序列，对消息体做流式加解密。
- 在 MSG 命令头部中加入 Encryption：算法名标识此消息已加密
- 要求：
 - 加密后的消息体在 Wireshark 抓包中不可直接读取明文
 - 使用同一条明文消息对比未加密与加密两次抓包；加密抓包的 body 中不得出现该明文原文
 - 加密消息和未加密消息能共存（通过头部标识区分）
 - 加解密过程不影响聊天功能的正常使用
 - 若接收方在 LOGIN 时未提交 Key 头部（即接收方不使用加密），则发送方不能向该用户发送加密消息；服务器应当拒绝并返回 400 BAD_REQUEST

4.4.3 E3: 多媒体消息 (10 分)

- 扩展 CHAT/1.0 协议支持非文本数据的传输，包含以下两个递进层次：
 - 文件传输 (6 分): 新增 FILE 命令，支持在聊天中发送文件（仅限已登录用户使用，未登录返回 401 NOT_LOGGED_IN）。
 - * 协议格式:


```
FILE CHAT/1.0\r\n
Filename: photo.jpg\r\n
Content-Type: image/jpeg\r\n
Content-Length: 524288\r\n
\r\n
[二进制数据...]
```
 - * 发送端读文件 → 通过 socket 发送二进制数据
 - * 接收端完全依赖 Content-Length 确定 body 边界（二进制数据中可能出现 \r\n\r\n 模式，不能用作结束判断）
 - * 接收后将文件保存到本地磁盘
 - * 报告中需给出发送文件大小与接收文件大小一致的验证截图或哈希值
 - 表情 (4 分): 在文本消息中支持表情符号。
 - * 定义表情映射表，如 :smile: → “笑脸”、:sad: → “难过”、:like: → “点赞”
 - * 消息发送时不做特殊处理，接收方显示时解析替换

💡 提示：选做扩展任务的同学，请将每项扩展功能的测试结果截图保存（如聊天界面截图、Wireshark 抓包截图等），用于报告撰写（参见 6.2.1 节）。

5 开发与运行环境

5.1 软件要求

- 操作系统：不限（Windows / macOS / Linux 均可）
- Python 3.8+（推荐 3.10 或更高版本）
- Wireshark——最新稳定版即可

5.2 库使用规范

- 允许使用的库: 必做部分的网络通信层仅限 Python 标准库, 包括但不限于: socket, threading, os, sys, pathlib, datetime, hashlib, tkinter 等

- **禁止使用的库：**在必做部分中，禁止使用任何封装了网络通信或聊天服务器功能的第三方库，包括但不限于：Twisted, websockets, Flask-SocketIO, requests。所有 CHAT/1.0 通信必须基于 socket 标准库手动完成。
- 图形界面与扩展任务的特殊情况：
 - GUI 展示层（窗口、按钮、图片显示）可以使用第三方库，如 PyQt6、Pillow 等
 - 若选择 Web 前端方案，需要在后端编写额外的 HTTP 接口或 WebSocket 桥接层，该桥接层仍须由学生自己实现协议转换逻辑，不得直接使用现成聊天室或即时通信框架
 - CHAT/1.0 协议解析、消息边界处理、状态管理和转发逻辑无论何种方案均须由学生自己实现

5.3 实验说明

- 服务器绑定 127.0.0.1 地址，客户端通过 localhost 或 127.0.0.1 访问。服务器端口在代码中自行指定（如 8080、8888 等未被占用的端口即可）。Wireshark 抓包选择 loopback（回环接口）进行捕获。
- 客户端与服务器的通信基于自定义的 CHAT/1.0 协议，协议规范参见附录 A。

6 答辩与报告

课程总成绩由报告（占 40%）和答辩（占 60%）两部分构成。根据所选做的内容，报告会对对应的源代码和文档打分，答辩会依据现场演示和提问回答情况打分。

6.1 演示内容

答辩时（每人约 10 分钟），学生需现场演示以下内容：

6.1.1 必选演示（约 6 分钟）

答辩时将按以下清单逐项检查评分。建议你提前对照此清单自检，确保每项均可现场复现。

1. 基本功能演示（约 2 分钟）

准备三个客户端（alice、bob、charlie）。

(a) 启动服务器，日志显示监听地址和端口

- (b) alice 以合法用户名登录 → 200 OK
- (c) bob 登录 → bob 收到 200 OK; 同时 alice 收到 201 USERLIST 推送 (登录触发推送)
- (d) alice 发送广播 MSG → alice 收到 200 OK; bob 收到 202 MSG_RELAYED (含 From: alice)
- (e) bob 查询 LIST → 返回 201 USERLIST, body 包含当前所有在线用户
- (f) charlie 以已在线用户名 alice 尝试登录 → 409 USERNAME_TAKEN (重复用户名拒绝)
- (g) charlie 未登录直接发送 MSG → 401 NOT_LOGGED_IN
- (h) charlie 发送未知命令、错误版本或非法 Content-Length → 400 BAD_REQUEST
- (i) alice 退出登录 → 200 OK; bob 收到更新后的 201 USERLIST 推送 (登出触发推送)
- (j) 运行测试 2 脚本验证半包处理 (一条请求分多次发出 → 接收方仍收到完整消息), 运行测试 3 脚本验证粘包处理 (多条请求一次发出 → 每条独立处理、不合并)

2. 私聊与群聊演示 (约 2 分钟)

三个客户端 alice、bob、charlie 全部登录后依次操作。

- (a) alice、bob、charlie 均已完成登录
——私聊验证——
- (b) alice 发送 MSG, 头部为 To: bob → alice 收到 200 OK; bob 收到 202 MSG_RELAYED; charlie 确认未收到任何消息 (私聊只到达目标)
- (c) alice 发送 MSG, 不带 To 头部 → bob 和 charlie 均收到 202 MSG_RELAYED (缺少 To 时退化为全体广播)
- (d) alice 发送 MSG, 头部为 To: nonexistent → 404 USER_NOT_FOUND (目标用户不存在)
——群组验证——
- (e) alice 创建群组 room1 → 200 OK; alice 自动成为成员
- (f) alice 再次创建同名群组 room1 → 409 GROUP_EXISTS
- (g) bob 加入群组 room1 → 200 OK
- (h) bob 加入不存在的群组 → 404 GROUP_NOT_FOUND
- (i) 新开一个客户端 (未登录) 请求加入 room1 → 401 NOT_LOGGED_IN

- (j) alice 向群组 room1 发消息 → alice 收到 200 OK; bob (成员) 收到 202 MSG_RELAYED; charlie (非成员) 确认未收到
- (k) charlie(未加入 room1)向 room1 发消息 → 服务器拒绝并返回 403 NOT_GROUP_MEMBER

3. Wireshark 分析展示 (约 1 分钟)

- 展示三次握手的三个包 (SYN、SYN-ACK、ACK), 标注 seq/ack 变化, 说明三次握手的目的
- 展示 LOGIN、MSG 请求报文和 200 OK、202 MSG_RELAYED 响应/推送报文, 标注命令行 (状态行)、头部、空行和 body, 并能对应到代码中的解析或构造位置
- 选取一段连续数据传输过程, 计算 payload 字节数, 验证 seq 增量与传输数据量一致
- 展示连接终止相关 FIN/ACK 包, 说明 TCP 全双工关闭需要双方分别发送 FIN

6.1.2 扩展功能演示 (约 2-4 分钟)

根据实际选做情况演示对应项, 建议按以下步骤串联。

1. E1 图形化客户端 (选做)

展示 GUI 底层真正调用了自己实现的 CHAT/1.0 协议代码, 而非静态界面。

- (a) 启动服务器, 打开第一个 GUI 客户端, 输入服务器地址、端口和用户名 alice, 点击登录 → 登录成功, 聊天主界面出现
- (b) 确认主界面包含: 消息显示区、消息输入框、发送按钮、在线用户列表
- (c) 打开第二个 GUI 客户端, 以 bob 登录 → alice 端的在线用户列表自动更新为两人
- (d) alice 输入一条广播消息并发送 → bob 的消息显示区出现该消息
- (e) bob 回复 → alice 也能收到

2. E2 消息加密 (选做)

展示加密消息的收发全过程, 并用 Wireshark 对比加密前后的 body。

- (a) 服务器和两个客户端 alice、bob 均按加密方案启动或登录 (LOGIN 时携带密钥)
- (b) alice 发一条明文消息 (如 Hello), bob 收到并正常显示

- (c) 打开 Wireshark，抓取一次加密传输：alice 发同一条消息 → Wireshark 中 body 显示为乱码，不得出现明文
- (d) 对比同一条消息的未加密抓包（body 可读）和加密抓包（body 不可读），证明加密生效
- (e) 切换一个未加密客户端 charlie 登录，与 alice 收发消息 → 加密和未加密消息共存，接收端通过头部 Encryption 标识正确识别

3. E3 多媒体消息（选做）

展示文件传输（含大小/哈希验证）和表情替换。

- (a) alice 通过 FILE 命令发送一个文件（如 photo.jpg），包含 Filename、Content-Type、Content-Length 头部
- (b) 接收端将接收到的数据保存到本地磁盘
- (c) 对比发送端和接收端的文件大小、并计算哈希值 → 两者均一致
- (d) 在文本消息中使用表情符号，如 :smile:、:sad:、:like: → 接收方显示时自动替换为对应的表情图案

6.2 报告

6.2.1 报告内容

报告应包含以下内容，报告中所有截图需配有文字说明，解释截图中的关键信息。

1. L1: CHAT/1.0 核心服务器实现

- 服务器整体架构说明，核心函数及功能
- CHAT/1.0 协议解析流程说明，附关键代码片段（协议格式参见附录 A.1）
- 消息边界处理的实现方式说明
- 用户状态管理的实现方式说明
- 测试验证截图：
 - 分别测试四种命令的响应
 - 多消息连续发送验证消息边界处理
 - 异常处理验证（未登录拒绝、重复用户名）

2. L2: 私聊与群聊

- 私聊协议扩展说明（To 头部、目标不存在的响应）
- 群组命令说明（创建群组、加入群组、群组消息发送）

- 消息路由规则说明（哪些用户应收到，哪些用户不应收到）
- 测试验证截图：
 - 私聊目标用户收到、非目标用户未收到的对照截图
 - 群聊成员收到、非成员未收到的对照截图

3. L3: Wireshark 抓包分析

- TCP 三次握手截图及分析——标注 seq/ack 变化，说明为什么是三次握手
- CHAT 请求/响应报文截图及分析——标注关键字段
- TCP 序列号追踪分析——展示 seq/ack 递增关系，验证传输数据量
- TCP 四次挥手截图及分析——标注挥手过程，说明为什么是四次而非两次

4. 扩展任务（选做，按实际完成情况撰写）

- 每个选做的扩展任务均需包含：实现方案说明 + 测试验证截图

5. 实验总结

6.2.2 提交要求

- 提交材料：将源代码（server.py、client.py 等，需包含清晰注释）与报告（PDF 格式）打包为一个压缩包
- 命名规范：压缩包以 学号-姓名.zip 命名
- 提交地址：<https://yunpan.bjfu.edu.cn:443/link/4270BB8B137EE02440B411B8550CC681>
密码：vuZx
- 截止日期：2026 年 7 月 12 日

7 参考资料

1. James F. Kurose, Keith W. Ross. *Computer Networking: A Top-Down Approach*, 8th Ed. 第 2 章（Socket 编程），第 3 章（TCP）
2. Python 官方文档—socket 模块：<https://docs.python.org/3/library/socket.html>
3. Python 官方文档—threading 模块：<https://docs.python.org/3/library/threading.html>
4. Python 官方文档—tkinter 模块：<https://docs.python.org/3/library/tkinter.html>

5. Wireshark 官方文档—抓取本地回环流量:<https://wiki.wireshark.org/CaptureSetup/Loopback>

A 附录 A: CHAT/1.0 协议参考

本附录定义了课设需要的 CHAT/1.0 协议子集。L1 和 L2 为必做协议内容；扩展任务如需增加协议字段或命令，须在报告中完整说明。

A.1 CHAT/1.0 协议格式

CHAT/1.0 是一个基于文本的应用层协议，格式与 HTTP/1.1 类似，由命令行、头部字段、空行和消息体组成。

请求报文格式（客户端 → 服务器）：

其中 SP 表示一个 ASCII 空格字符（0x20），CRLF 表示 \r\n。

```
LOGIN CHAT/1.0\r\n
User: alice\r\n
Content-Length: 0\r\n
\r\n
```

组成部分	说明
命令行	COMMAND SP CHAT/1.0 CRLF
头部字段	每行一个 字段名: 值, 以 \r\n 结束
空行	单独的 \r\n, 标记头部结束
消息体	可选, 由 Content-Length 头部指定长度

支持的请求命令：

分组	命令	功能	必选头部
L1	LOGIN	用户登录, 设置用户名	User, Content-Length
L1	MSG	发送聊天消息; L2 中可通过 To 头部指定目标	Content-Length
L1	LIST	查询在线用户列表	Content-Length
L1	LOGOUT	退出登录	Content-Length
L2	CREATEGROUP	创建群组	Group, Content-Length
L2	JOINGROUP	加入群组	Group, Content-Length

响应报文格式（服务器 → 客户端）：

```
CHAT/1.0 200 OK\r\n
Content-Length: 0\r\n
\r\n
```

组成部分	说明
状态行	CHAT/1.0 SP STATUS_CODE SP REASON_PHRASE CRLF
响应头部	每行一个头部，以 \r\n 结束
空行	单独的 \r\n，标记头部结束
消息体	可选，由 Content-Length 指定长度

服务器转发报文（服务器 → 客户端的非请求消息）：

当其他用户发送消息时，服务器会主动向在线用户推送：

```
CHAT/1.0 202 MSG_RELAYED\r\n
From: alice\r\n
Content-Length: 12\r\n
\r\n
Hello world!
```

此外，当有用户登录或登出时，服务器也会向所有仍在线用户推送更新后的用户列表：

```
CHAT/1.0 201 USERLIST\r\n
Content-Length: 17\r\n
\r\n
alice,bob,charlie
```

LIST 和用户状态变化推送使用相同的 201 USERLIST 报文格式；区别只是触发方式不同：前者由客户端主动请求触发，后者由登录、登出或异常断开事件触发。

A.2 必须实现的头部字段

分组	头部字段	出现位置	说明
必做	User	LOGIN 请求	用户名
必做	Content-Length	请求/响应/推送	消息体长度（字节数）
必做	From	202 推送	消息发送者
必做 L2	To	MSG 请求	消息接收者（私聊或群聊时使用）
必做 L2	Group	CREATEGROUP 或 JOINGROUP 请求	群组名称
扩展 E2	Key	LOGIN 请求	加密密钥
扩展 E2	Encryption	MSG 请求	加密算法标识
扩展 E3	Filename	FILE 请求/推送	文件名
扩展 E3	Content-Type	FILE 请求/推送	文件 MIME 类型，如 image/jpeg

A.3 必须实现的状态码

分组	状态码	原因短语	使用场景
必做	200	OK	请求成功
必做	201	USERLIST	LIST 命令的响应, 或用户登录/登出/异常断开时服务器主动推送的用户列表更新
必做	202	MSG_RELAYED	服务器转发的聊天消息
必做	400	BAD_REQUEST	协议格式错误、命令无法解析、必要头部缺失或非法
必做	401	NOT_LOGGED_IN	未登录状态下发送了非 LOGIN 命令
必做	409	USERNAME_TAKEN	登录时用户名已被占用
必做	500	INTERNAL_ERROR	服务器内部错误
必做 L2	404	USER_NOT_FOUND	私聊目标用户不存在
必做 L2	404	GROUP_NOT_FOUND	加入或发送到不存在的群组
必做 L2	403	NOT_GROUP_MEMBER	发送者不是目标群组成员
必做 L2	409	GROUP_EXISTS	创建已存在的群组

A.4 关键实现细节

1. 行结束符: 协议规范要求使用 `\r\n` (CRLF) 作为行结束符。仅使用 `\n` (LF) 可能导致解析错误。
2. 空行判定: 头部结束的标志是连续两个 CRLF: `\r\n\r\n`。这是你判断”头部解析完毕”的唯一可靠依据。
3. TCP 是流式协议: 一次 `recv()` 可能收到多个完整的请求 (粘包), 也可能只收到半个请求 (半包)。你的服务器必须能通过 `\r\n\r\n` 和 `Content-Length` 判断一条 CHAT/1.0 报文是否完整, 不能假设一次 `recv()` 刚好对应一条请求。
4. 消息转发机制: 聊天室服务器在收到某个用户的 MSG 后, 需要向其他所有已登录用户发送 202 MSG_RELAYED 报文。这意味着:
 - 服务器需要维护所有已登录用户的连接列表

- 转发消息时，遍历列表向每个用户发送
- 注意不要将消息回发给发送者本人

5. 用户状态管理：

- 每个连接需要记录其当前状态（未登录/已登录）
- 建议为每个连接维护一个客户端对象，包含 socket、地址、用户名、状态
- 登录流程：收到 LOGIN → 检查用户名是否可用 → 设置状态为已登录
- 断开流程：客户端断开 → 从在线列表移除 → 向所有仍在线用户推送更新后的 201 USERLIST → 后续 LIST 查询不再返回该用户名

6. 并发状态一致性：服务器需要维护在线用户表、群组表等共享状态。

- 若采用多线程处理客户端连接，对这些共享状态的读写必须进行同步保护
- 若采用其他并发模型，也必须保证用户名检查、登录、退出、广播遍历和群组操作时状态一致

A.5 Wireshark 抓包基本步骤

1. 启动 Wireshark, 选择捕获接口：macOS 选 lo0 (Loopback), Linux 选 lo, Windows 选 Npcap Loopback Adapter
2. 设置过滤规则：tcp.port == 8888, 只显示与你的服务器相关的流量
3. 启动捕获
4. 在你的服务器和客户端之间进行一次完整的 CHAT/1.0 交互（LOGIN → 发消息 → LOGOUT）
5. 停止捕获
6. 在 Wireshark 中逐包分析，可右键选择 Follow → TCP Stream 查看完整的应用层数据
7. 对关键数据包截图，添加到你的分析报告中

关键过滤表达式：

过滤表达式	作用
tcp.port == 8888	只显示 8888 端口的流量
tcp.flags.syn == 1	只显示 SYN 标志位的数据包（三次握手）
tcp.flags.fin == 1	只显示 FIN 标志位的数据包（四次挥手）